

EduGenie

Google Gemini Powered Learning Assistant

Phase 5 – Project Development

Implementation, Source Code Structure, AI Integration and Application Development

Project Development Report

Table of Contents

1. Introduction
2. Development Environment
3. Project Structure
4. Application Architecture During Development
5. Frontend Development
6. Backend Development
7. AI Integration
8. Q&A; Module Implementation
9. Explanation Module Implementation
10. Quiz Module Implementation
11. Summary Module Implementation
12. Learning Recommendation Module
13. API Endpoint Implementation
14. Frontend–Backend Integration
15. Environment Configuration
16. Security Implementation
17. Application Execution
18. Development Summary
19. Conclusion

1. Introduction

This document presents Phase 5 – Project Development for EduGenie, a Google Gemini powered learning assistant. Phase 5 documents the actual development and implementation of the application, converting the requirements and design produced in earlier phases into a working software system.

The development described in this report covers the following areas, each verified directly against the project's source code package (**EduGenie-Ai01-main.zip**):

- Frontend implementation using HTML, CSS and JavaScript
- Backend implementation using the FastAPI framework
- Implementation of five learning modules: question answering, explanation, quiz generation, summarization and learning-path recommendation
- Integration with the Google Gemini API for AI-generated content
- Implementation of REST API endpoints
- Communication between the frontend and backend
- Environment-based configuration
- Steps required to run the application

All technical details in this report were verified against the actual source files in the project. Where the source code did not confirm a particular implementation detail, this is stated explicitly rather than assumed.

2. Development Environment

The development environment was identified directly from **requirements.txt**, **README.md** and the source files. The project does not include a local/offline AI model; all five learning modules call the Google Gemini API through a single shared client module.

Category	Technology / Detail
Programming Language	Python (backend); HTML, CSS and JavaScript (frontend)
Backend Framework	FastAPI (fastapi>=0.115,<1.0), served with Uvicorn (uvicorn[standard])
Templating	Jinja2 (jinja2>=3.1,<4.0), via FastAPI's Jinja2Templates
Frontend	Static HTML template (index.html), style.css, app.js (vanilla JavaScript, fetch API)
AI Model / API	Google Gemini, via the google-genai SDK (google-genai>=1.30,<2.0). Default model identifier: gemini-2.5-flash (configurable via the GEMINI_MODEL environment variable)
Local Model	This implementation detail is not specified in the available source code. No local/offline model (e.g. LaMini-Flan-T5) is present in the source; every module uses the Gemini API exclusively.
Configuration	python-dotenv (python-dotenv>=1.0,<2.0), loading variables from a .env file via load_dotenv()
Data Validation	Pydantic (pydantic>=2.7,<3.0), used for FastAPI request models

Category	Technology / Detail
Dependency File	requirements.txt
Testing Tool	pytest, configured via pytest.ini (testpaths = tests); FastAPI TestClient used in tests/test_api.py

Table 1: Development environment, compiled from requirements.txt, .env.example and source files.

3. Project Structure

The actual folder structure of the EduGenie-Ai01-main package is reproduced below.

```

EduGenie-Ai01-main/
├── .env.example
├── .gitignore
├── README.md
├── pytest.ini
├── requirements.txt
├── backend/
│   ├── __init__.py
│   ├── main.py
│   ├── gemini_client.py
│   ├── services/
│   │   ├── __init__.py
│   │   ├── qna.py
│   │   ├── explanation_module.py
│   │   ├── quiz_module.py
│   │   ├── summary_module.py
│   │   └── learning_path.py
│   └── frontend/
│       ├── static/
│       │   ├── app.js
│       │   └── style.css
│       ├── templates/
│       └── index.html
├── tests/
└── test_api.py

```

Figure showing the verified directory tree of the source package.

File / Folder	Purpose (verified from source)
backend/main.py	FastAPI application entry point; defines the app instance, mounts static files, configures Jinja2 templates, declares Pydantic request models and all API routes.
backend/gemini_client.py	Shared client module that creates a cached google-genai Client, reads GEMINI_MODEL / GEMINI_API_KEY from the environment, and exposes an async generate_text() helper used by every learning module.
backend/services/qna.py	Implements the question-answering module; builds a prompt and calls generate_text().
backend/services/explanation_module.py	Implements the topic-explanation module using a structured beginner-friendly prompt.

File / Folder	Purpose (verified from source)
backend/services/quiz_module.py	Implements MCQ quiz generation, including JSON extraction/cleanup (clean_json_block) and validation of the returned quiz (validate_quiz).
backend/services/summary_module.py	Implements passage summarization with a revision-oriented prompt.
backend/services/learning_path.py	Implements personalized learning-path/recommendation generation based on topic, level and weekly study hours.
frontend/templates/index.html	Jinja2 HTML template rendered at the root route; contains the task selector, input form and result panel.
frontend/static/app.js	Client-side JavaScript: builds the request body based on the selected task, calls the corresponding backend endpoint with fetch(), and renders the response (including a dedicated quiz renderer).
frontend/static/style.css	Styling for the interface (layout, colors, cards, ambient background effects).
requirements.txt	Declares backend dependencies: fastapi, uvicorn[standard], jinja2, python-dotenv, pydantic, google-genai.
.env.example	Template for required environment variables: GEMINI_API_KEY and the optional GEMINI_MODEL.
pytest.ini	Pytest configuration; sets testpaths = tests.
tests/test_api.py	Automated tests using FastAPI's TestClient: health check, home page load, and validation of an empty question on /qa.

Table 2: Role of each source file, determined from direct inspection of the code.

4. Application Architecture During Development

The source code confirms a simple, single-service architecture with no database and no authentication layer. The FastAPI backend serves both the HTML interface and the JSON API, and every learning module delegates directly to the Gemini API through `gemini_client.py`.

```
[ User (Browser) ]
|
v
[ HTML / CSS / JavaScript Frontend (index.html, style.css, app.js) ]
| fetch() POST request (JSON)
v
[ FastAPI Backend (backend/main.py) ]
| route dispatch based on endpoint
v
[ Selected Learning Module (qna.py / explanation_module.py /
quiz_module.py / summary_module.py /
learning_path.py) ]
| prompt string
v
[ gemini_client.py -> Google Gemini API (gemini-2.5-flash) ]
| generated text (or JSON for quizzes)
v
[ Learning Module -> FastAPI Response {"success": true, "result": ...} ]
|
v
[ Frontend JavaScript -> Result Panel ]
|
v
[ User (Browser) ]
```

Figure 1: EduGenie Development Architecture (verified from `backend/main.py`, `gemini_client.py` and the service modules).

No database, user-account system, authentication mechanism, or external deployment infrastructure is present in the source code. The application is a stateless request/response system: each request is processed independently and no learner data is persisted.

5. Frontend Development

5.1 HTML Structure

`index.html` (rendered via Jinja2 from `backend/main.py`) defines a single-page interface consisting of a header/hero section, a “workspace” card containing a task `<select>` element (Ask a Question, Explain a Topic, Generate Quiz, Summarize Text, Recommend Learning Path), a dynamic “extraFields” container, an input `<textarea>`, a submit button, and a result card with a status indicator and a result display area.

5.2 CSS Implementation

`style.css` implements a dark, gradient-based theme (navy/purple background with teal and violet accent blobs), a centered container of fixed maximum width, rounded “card” panels with a translucent background and backdrop blur, and responsive typography using `clamp()` for the hero heading. Layout is achieved with flexbox and CSS grid (for the level/hours fields in the learning-path form).

5.3 JavaScript Implementation

app.js implements the following client-side behavior, confirmed from source:

- **Task selection:** updateForm() changes the input label, placeholder text and any extra fields shown, depending on the selected task (e.g. a question-count field for quizzes, or level/hours fields for learning-path recommendations).
- **Input handling:** the textarea value is read and trimmed before submission; an empty input is rejected client-side with a message.
- **API requests:** submit() builds the request body appropriate to the selected task and sends it with fetch() as a POST request with a JSON body to the matching endpoint (/qa, /explain, /quiz, /summarize or /learn/recommendations).
- **Response handling:** the JSON response is parsed; a non-OK HTTP status throws an error using the detail field returned by the backend.
- **Result display:** for the quiz task, renderQuiz() renders each question, its four options, the correct answer and, if present, an explanation; for all other tasks the raw text result is shown directly in the result panel.
- **Error handling:** fetch/network or backend errors are caught and displayed in the result panel prefixed with "Error:", and the status indicator is updated to "Error".
- **Keyboard shortcut:** Ctrl/Cmd + Enter inside the textarea triggers submission.

No screenshots are included in this report, because the application could not be executed in the current environment (it requires a Google Gemini API key). In keeping with the accuracy requirements for this document, fictional or simulated screenshots have been omitted rather than fabricated.

6. Backend Development

The backend is implemented entirely in **backend/main.py**, which is the main FastAPI application file. It performs the following, verified directly from the source:

- **Application initialization:** creates the FastAPI app with a title, version ("1.0.0") and description; calls load_dotenv() to load environment variables.
- **Static and template configuration:** mounts frontend/static at /static via StaticFiles, and configures Jinja2Templates pointing at frontend/templates.
- **Routes:** GET / (renders index.html), GET /health (basic health check), and POST routes for /qa, /explain, /quiz, /summarize and /learn/recommendations.
- **Request handling:** each POST route accepts a Pydantic request model (TextRequest, QuestionRequest, QuizRequest or LearningPathRequest) which FastAPI validates automatically.
- **Module integration:** each route awaits the corresponding service function (e.g. answer_question(), explain_topic(), generate_quiz(), summarize_text(), get_learning_recommendations()) and wraps the result as {"success": true, "result": ...}.
- **Error handling:** a shared service_error() helper logs the exception with logger.exception() and raises an HTTPException with status code 502 and a generic, safe error message, so that internal details (such as provider errors) are not leaked to the client.
- **Configuration:** no middleware, authentication layer or database session handling is present in the source code.

Below is a short representative snippet from backend/main.py showing route registration and the shared error handler:

```
def service_error(exc: Exception) -> HTTPException:
    logger.exception("EduGenie request failed", exc_info=exc)
    return HTTPException(
        status_code=502,
        detail="The learning service is unavailable. Check your Gemini API key and try again.",
    )

@app.post("/qa")
async def qa(payload: QuestionRequest):
    try:
        return {"success": True, "result": await answer_question(payload.question)}
    except Exception as exc:
        raise service_error(exc)
```

Source: backend/main.py

7. AI Integration

All AI generation in EduGenie is handled by a single module, **backend/gemini_client.py**, which every learning module imports and calls. There is no local or offline AI model in the source code; all five modules use the Google Gemini API exclusively.

7.1 Client Initialization

`get_client()` is decorated with `functools.lru_cache(maxsize=1)`, so the Gemini client is constructed once and reused. It reads the API key from the `GEMINI_API_KEY` environment variable (falling back to `GOOGLE_API_KEY` if present) and raises a `RuntimeError` if no key is configured.

7.2 Model Identifier

The exact model identifier used is **gemini-2.5-flash**. It is read from the `GEMINI_MODEL` environment variable, defaulting to "gemini-2.5-flash" if the variable is not set (`MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")`). This is confirmed in both `gemini_client.py` and `.env.example`. The source code does not reference Gemini 1.5 Pro or any other model.

7.3 Request Construction and Response Handling

`generate_text()` is an async function that accepts a prompt string, an optional temperature (default 0.3) and an optional `max_output_tokens` (default 2048). It calls `client.aio.models.generate_content()` with the configured model name, the prompt as contents, and a `types.GenerateContentConfig` carrying the temperature and `max_output_tokens`. The response's `.text` attribute is extracted and stripped; if it is empty, a `RuntimeError` is raised.

```
MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")

@lru_cache(maxsize=1)
def get_client():
    api_key = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")
    if not api_key:
        raise RuntimeError(
            "Gemini API key is missing. Set GEMINI_API_KEY in the .env file."
        )
    return genai.Client(api_key=api_key)

async def generate_text(prompt, *, temperature=0.3, max_output_tokens=2048) -> str:
    client = get_client()
    response = await client.aio.models.generate_content(
        model=MODEL_NAME,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=temperature,
            max_output_tokens=max_output_tokens,
        ),
    )
    text = getattr(response, "text", None)
    if not text:
        raise RuntimeError("Gemini returned an empty response.")
    return text.strip()
```

Source: backend/gemini_client.py

This implementation detail is not specified in the available source code: no local model such as LaMini-Flan-T5-783M is loaded, imported, or referenced anywhere in the project. Every learning module, including the explanation module, calls `generate_text()` and therefore uses the Gemini API.

8. Q&A Module Implementation

File: backend/services/qna.py

Endpoint: POST /qa **Function:** `answer_question(question)`

The module builds a single prompt instructing the model to act as “EduGenie, a student-friendly educational assistant,” to answer accurately and concisely, to state any assumption if the question is ambiguous, and not to invent sources or statistics. It calls `generate_text()` with temperature 0.2 and `max_output_tokens` 1200, and returns the resulting text.

```
Question -> POST /qa -> qna.answer_question() -> gemini_client.generate_text()
-> Gemini API (gemini-2.5-flash) -> Answer text -> JSON response
```

Implementation flow, confirmed from backend/main.py and backend/services/qna.py.

Representative snippet:

```
async def answer_question(question: str) -> str:
    prompt = f"""
    You are EduGenie, a student-friendly educational assistant.
    Answer the student's question accurately and concisely.
    ...
    Student question:
    {question}
    """
    return await generate_text(prompt, temperature=0.2, max_output_tokens=1200)
```

Source: backend/services/qna.py (prompt body abbreviated for brevity).

9. Explanation Module Implementation

File: backend/services/explanation_module.py

Endpoint: POST /explain **Function:** `explain_topic(topic)`

The explanation module constructs a beginner-oriented prompt requesting a simple definition, an explanation of how the concept works, an easy example, and key points to remember. It calls `generate_text()` with temperature 0.25 and `max_output_tokens` 1200. As noted in Section 7, this module uses the Gemini API, not a local model — no local model is present in the source code.

```
async def explain_topic(topic: str) -> str:
    prompt = f"""
    Explain the following educational topic as if you are teaching a beginner.
    Topic:
    {topic}
    Format:
    1. Simple definition
    2. How it works
    3. Easy example
    4. Key points to remember
    """
```

```
return await generate_text(prompt, temperature=0.25, max_output_tokens=1200)
```

Source: backend/services/explanation_module.py.

10. Quiz Module Implementation

File: backend/services/quiz_module.py

Endpoint: POST /quiz **Functions:** generate_quiz(text, count), clean_json_block(text), validate_quiz(data)

generate_quiz() builds a prompt asking Gemini to return exactly count multiple-choice questions as a raw JSON array (no Markdown), where each item has a question, exactly four options, one answer matching an option, and a short explanation. The QuizRequest Pydantic model (in backend/main.py) constrains count to between 1 and 10, confirming the “1–10 question MCQ generation” feature described in the README.

The raw text returned by Gemini is passed through clean_json_block(), which strips Markdown code fences and isolates the JSON array using the first '[' and last ']' characters. The result is parsed with json.loads() and passed to validate_quiz(), which discards any item that is not a dictionary, lacks a non-empty question, does not have exactly four options, or has no usable answer. If the answer field is numeric (1–4) it is converted to the corresponding option text. If no valid questions remain, a ValueError is raised.

```
Topic / Passage + Count -> POST /quiz -> quiz_module.generate_quiz()
-> gemini_client.generate_text() -> Gemini API -> raw text
-> clean_json_block() -> json.loads() -> validate_quiz()
-> list[dict] -> JSON response -> renderQuiz() (frontend)
```

Implementation flow, confirmed from backend/main.py, quiz_module.py and app.js.

Representative snippet (validation logic):

```
def validate_quiz(data: Any) -> list[dict]:
    if not isinstance(data, list):
        raise ValueError("Quiz response must be a JSON list.")
    cleaned = []
    for item in data:
        ...
        if not question or len(options) != 4 or not answer:
            continue
        if answer not in options:
            if answer.isdigit() and 1 <= int(answer) <= 4:
                answer = options[int(answer) - 1]
            else:
                continue
        cleaned.append({"question": question, "options": options,
            "answer": answer, "explanation": ...})
    if not cleaned:
        raise ValueError("No valid quiz questions were returned.")
    return cleaned
```

Source: backend/services/quiz_module.py (abbreviated).

Answer checking against a stored key, quiz scoring, and quiz persistence are not implemented in the source code; the answer for each question is returned directly in the API response alongside the question and options, and grading/interaction beyond display is handled only by the frontend's renderQuiz() function, which shows the correct answer and explanation immediately.

11. Summary Module Implementation

File: backend/services/summary_module.py

Endpoint: POST /summarize **Function:** summarize_text(text)

The summary module constructs a prompt instructing Gemini to summarize the supplied educational passage for quick revision: preserving important facts, removing repetition, using simple language, using headings and bullet points where useful, and not introducing information absent from the passage. It calls generate_text() with temperature 0.2 and max_output_tokens 1600.

```
async def summarize_text(text: str) -> str:
    prompt = f"""
    Summarize the following educational passage for quick revision.
    Requirements:
    - Keep the important facts and ideas.
    - Remove repetition and unnecessary wording.
    - Use simple language.
    - Use headings and bullet points when useful.
    - Do not introduce information that is not present in the passage.
    Passage:
    {text}
    """
    return await generate_text(prompt, temperature=0.2, max_output_tokens=1600)
```

Source: backend/services/summary_module.py.

12. Learning Recommendation Module

File: backend/services/learning_path.py

Endpoint: POST /learn/recommendations **Function:** get_learning_recommendations(topic, level, weekly_hours)

The LearningPathRequest model (backend/main.py) accepts a topic, a level (defaulting to “beginner”), and weekly_hours (default 5, constrained between 1 and 40). The prompt asks Gemini to produce a personalized learning path organized from beginner to advanced, including learning stages, concepts per stage, a suggested timeline, practice ideas, resource types (videos, articles, books, documentation), a weekly routine, and a milestone per stage. It explicitly instructs the model not to invent specific URLs. The call uses temperature 0.35 and max_output_tokens 2400.

The recommendations are generated fresh for each request based only on the parameters supplied in that request. The source code does not implement storage of user learning history or any personalization based on past sessions; each recommendation is generated independently.

```
async def get_learning_recommendations(topic, level="beginner", weekly_hours=5) -> str:
    prompt = f"""
    Create a personalized learning path for the topic "{topic}".
    Learner level: {level}
    Available study time: {weekly_hours} hours per week
    ...
    """
    return await generate_text(prompt, temperature=0.35, max_output_tokens=2400)
```

Source: backend/services/learning_path.py (abbreviated).

13. API Endpoint Implementation

The following endpoints are implemented in `backend/main.py`. All request bodies are validated by Pydantic models defined in the same file.

Endpoint	Method	Purpose	Source Module
/	GET	Renders and returns the main HTML interface (<code>index.html</code>)	<code>backend/main.py</code>
/health	GET	Basic health check; returns <code>{"status": "ok", "service": "EduGenie"}</code>	<code>backend/main.py</code>
/qa	POST	Answers a student question using Gemini	<code>backend/services/qa.py</code>
/explain	POST	Explains a topic in beginner-friendly terms using Gemini	<code>backend/services/explanation_module.py</code>
/quiz	POST	Generates a validated multiple-choice quiz from supplied text	<code>backend/services/quiz_module.py</code>
/summarize	POST	Summarizes an educational passage using Gemini	<code>backend/services/summary_module.py</code>
/learn/recommendations	POST	Generates a personalized learning path/recommendation	<code>backend/services/learning_path.py</code>

Table 3: API endpoints, verified directly from `backend/main.py` route decorators.

13.1 Request Models (verified from `backend/main.py`)

Model	Fields (as declared)	Used by
<code>TextRequest</code>	<code>text: str (min_length=1, max_length=20000)</code>	<code>/explain</code> , <code>/summarize</code>
<code>QuestionRequest</code>	<code>question: str (min_length=1, max_length=10000)</code>	<code>/qa</code>
<code>QuizRequest</code>	<code>text: str (min_length=1, max_length=20000)</code> ; <code>count: int (default=3, ge=1, le=10)</code>	<code>/quiz</code>
<code>LearningPathRequest</code>	<code>topic: str (min_length=1, max_length=500)</code> ; <code>level: str (default="beginner", max_length=50)</code> ; <code>weekly_hours: int (default=5, ge=1, le=40)</code>	<code>/learn/recommendations</code>

Table 4: Pydantic request schemas, reproduced exactly from `backend/main.py`.

Each successful response follows the shape `{"success": true, "result": ...}`, where `result` is a string for `/qa`, `/explain`, `/summarize` and `/learn/recommendations`, and a list of question objects for `/quiz`. On failure, the shared `service_error()` handler returns HTTP 502 with a generic detail message.

14. Frontend–Backend Integration

The frontend and backend communicate over plain HTTP JSON requests issued by the browser's fetch() API, with no additional client libraries or frameworks involved.

```

User Input (task selection + text in textarea)
|
v
Frontend JavaScript (app.js: submit())
| builds JSON body based on selected task
v
HTTP POST Request (fetch, Content-Type: application/json)
|
v
FastAPI Endpoint (/qa, /explain, /quiz, /summarize, /learn/recommendations)
| Pydantic validation
v
Learning Module (qna.py / explanation_module.py / quiz_module.py /
summary_module.py / learning_path.py)
|
v
gemini_client.generate_text() -> Google Gemini API (gemini-2.5-flash)
|
v
Backend JSON Response {"success": true, "result": ...}
|
v
Frontend JavaScript (renders text, or renderQuiz() for quizzes)
|
v
Result Display in the Browser

```

Figure 2: Frontend–Backend Integration Flow (verified from app.js and backend/main.py).

Error conditions are surfaced end-to-end: a non-OK HTTP response causes app.js to throw an error using the detail field from the JSON body (set by service_error() on the backend), and the error message is displayed in the result panel.

15. Environment Configuration

.env.example defines the environment variables the project expects. The actual file contents are reproduced below (no real key is present in the template):

```

# Required
GEMINI_API_KEY=your_gemini_api_key_here

# Optional: change the Gemini model without editing Python code.
GEMINI_MODEL=gemini-2.5-flash

```

Source: .env.example.

The README instructs developers to copy .env.example to .env and replace the placeholder value with a real Gemini API key. backend/main.py calls load_dotenv() at import time, and gemini_client.py reads GEMINI_API_KEY (or, as a fallback, GOOGLE_API_KEY) at runtime via os.getenv(). GEMINI_MODEL is optional; if unset, the code defaults to "gemini-2.5-flash".

16. Security Implementation

The source code and README confirm the following security-related practices. These are clearly distinguished below between what is actually implemented in code and what is a recommended practice stated in the documentation.

Implemented in code / configuration:

- The Gemini API key is supplied only through environment variables (GEMINI_API_KEY / GOOGLE_API_KEY), read server-side in `gemini_client.py`; it is never sent to or exposed in the frontend.
- `.gitignore` excludes `.env`, virtual-environment folders (`.venv/`, `venv/`), Python cache files, and other local artifacts from version control.
- Backend error responses use a generic message ("The learning service is unavailable...") instead of exposing internal exception details to the client; full details are only written to the server log via `logger.exception()`.
- Request bodies are validated by Pydantic models with explicit length and range constraints (e.g. `max_length`, `ge`, `le`), which limits malformed or oversized input reaching the AI calls.

Recommended practice (stated in README, not a code mechanism):

- The README states: "Never commit `.env` or an API key" and that "your Gemini API key stays on the server."
- `.env.example` is intended as a safe template for other developers to copy without exposing real credentials.

No authentication, user accounts, authorization, encryption at rest, rate limiting, or CSRF protection is implemented in the source code. This implementation detail is not specified beyond what is listed above.

17. Application Execution

The following setup and execution steps are reproduced from README.md, verified against the project structure and dependencies.

Setup:

```
py -3.10 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env
```

After copying the template, the developer opens `.env` and replaces `your_gemini_api_key_here` with an actual Gemini API key.

Run:

```
python -m uvicorn backend.main:app --reload
```

The application is then available at **`http://127.0.0.1:8000`**, with interactive API documentation (automatically generated by FastAPI) at **`http://127.0.0.1:8000/docs`**.

Basic verification (as documented in the README):

```
Invoke-RestMethod http://127.0.0.1:8000/health

Invoke-RestMethod -Method Post `
  -Uri http://127.0.0.1:8000/qa `
  -ContentType "application/json" `
  -Body '{"question":"What is photosynthesis?"}'
```

Source: README.md.

Automated tests exist in tests/test_api.py (run via pytest, configured through pytest.ini) covering the /health endpoint, the home page, and validation of an empty question submitted to /qa. Detailed test results and coverage analysis belong to Phase 6 – Project Testing and are not repeated here.

18. Development Summary

Component	Implementation (verified from source)
Frontend	Implemented: single-page HTML interface (Jinja2 template), CSS styling, and vanilla JavaScript handling task selection, API calls (fetch) and result rendering, including a dedicated quiz renderer.
Backend	Implemented: FastAPI application (backend/main.py) with static file mounting, Jinja2 templating, five POST endpoints, one GET page route and one health-check route.
Q&A Module	Implemented via backend/services/qna.py and the /qa endpoint, calling Gemini with temperature 0.2.
Explanation Module	Implemented via backend/services/explanation_module.py and the /explain endpoint, calling Gemini with temperature 0.25. No local model is used.
Quiz Module	Implemented via backend/services/quiz_module.py and the /quiz endpoint; includes JSON cleanup and validation logic (1–10 questions, 4 options each).
Summary Module	Implemented via backend/services/summary_module.py and the /summarize endpoint, calling Gemini with temperature 0.2.
Learning Recommendations	Implemented via backend/services/learning_path.py and the /learn/recommendations endpoint, parameterized by topic, level and weekly study hours.
AI Integration	Implemented via backend/gemini_client.py, a single cached Gemini client used by all five modules, using model gemini-2.5-flash by default (configurable).
Configuration	Implemented via python-dotenv and a .env.example template; GEMINI_API_KEY (required) and GEMINI_MODEL (optional).

Table 5: Development summary across all major components.

19. Conclusion

Phase 5 has documented the actual implementation of EduGenie as verified against its source code. The project was developed as a lightweight, single-service application: a Jinja2/HTML frontend styled with CSS and driven by vanilla JavaScript, communicating over JSON with a FastAPI backend. Five learning modules — question answering, topic explanation, quiz generation, summarization and personalized learning-path recommendations — are each implemented as a small service function that constructs a task-specific prompt and delegates generation to a single, shared Google Gemini client using the gemini-2.5-flash model.

The implementation does not include a database, user accounts, authentication, or a local/offline AI model; configuration is handled through environment variables with a documented .env.example template, and error handling returns safe, generic messages to the client while logging full details server-side. A small automated test suite (tests/test_api.py) exercises the health check, the home page and basic input validation.

This report has intentionally not claimed successful end-to-end testing, performance statistics, or a completion percentage, as these are outside the scope of Phase 5 and, where applicable, belong to Phase 6 – Project Testing.